

Azure Migration Plan - eCOM-AI

Overview

This document outlines the step-by-step process to host the eCOM-AI e-commerce application on Microsoft Azure.

Application Components

Component	Current State	Azure Target
.NET API	Local (localhost:5001)	Azure App Service
Angular Client	Local (localhost:4200)	Azure Static Web Apps
PostgreSQL	Docker (localhost:5432)	Azure Database for PostgreSQL
Redis	Docker (localhost:6379)	Azure Cache for Redis
AI Search	Local	Azure AI Search
OpenAI	Local configuration	Azure OpenAI / OpenAI API

Phase 1: Azure Resource Creation

1.1 Login and Resource Group

```
# Install Azure CLI if not installed
# https://docs.microsoft.com/en-us/cli/azure/install-azure-cli

# Login to Azure
az login

# Create resource group
az group create --name ecom-ai-rg --location eastus
```

1.2 Database (PostgreSQL)

```
# Create Flexible Server
az postgres flexible-server create \
  --name ecom-ai-db \
  --resource-group ecom-ai-rg \
  --sku-name Standard_B1ms \
  --tier Burstable \
  --admin-user appuser \
  --admin-password YourSecurePassword123! \
  --storage-size 5120

# Allow access from Azure services
az postgres flexible-server firewall-rule create \
  --name AllowAzureServices \
  --resource-group ecom-ai-rg \
  --server-name ecom-ai-db \
  --start-ip-address 0.0.0.0 \
  --end-ip-address 0.0.0.0
```

1.3 Redis Cache

```
# Create Redis cache
az redis create \
  --name ecom-ai-cache \
  --resource-group ecom-ai-rg \
  --sku Basic --vm-size c0

# Get Redis access keys
az redis list-keys \
  --name ecom-ai-cache \
  --resource-group ecom-ai-rg
```

1.4 App Service Plan

```
# Create Linux App Service plan
az appservice plan create \
  --name ecom-ai-plan \
  --resource-group ecom-ai-rg \
  --sku B1 \
  --is-linux
```

Phase 2: Configuration Updates

2.1 Create Production Settings

Create a new file: `API/appsettings.Production.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=ecom-ai-db.postgres.database.azure.com;Port=5432;User Id=appuser; Password=YOUR_PASSWORD;Database=skinet;SSL Mode=Require",
    "IdentityConnection": "Server=ecom-ai-db.postgres.database.azure.com;Port=5432;User Id=appuser; Password=YOUR_PASSWORD;Database=identity;SSL Mode=Require",
    "Redis": "ecom-ai-cache.redis.cache.windows.net:6380,password=YOUR_REDIS_KEY,ssl=True,abortConnect=False"
  },
  "Token": {
    "Key": "PROD_KEY_MUST_BE_AT_LEAST_32_CHARACTERS_LONG!",
    "Issuer": "https://ecom-ai-api.azurewebsites.net"
  },
  "ApiUrl": "https://ecom-ai-api.azurewebsites.net/Content/",
  "OpenAI": {
    "ApiKey": "YOUR_OPENAI_API_KEY",
    "BaseUrl": "https://api.openai.com/",
    "ChatModel": "gpt-4o-mini",
    "EmbeddingModel": "text-embedding-3-small"
  },
  "AzureSearch": {
    "Endpoint": "https://YOUR-SEARCH-SERVICE.search.windows.net",
    "ApiKey": "YOUR-AZURE-SEARCH-API-KEY",
    "IndexName": "products-index",
    "RagIndexName": "rag-index"
  }
}
```

2.2 Update Program.cs for Production

In `API/Program.cs`, ensure the production configuration is loaded:

```
var host = builder.Build();

// Use production settings if ASPNETCORE_ENVIRONMENT is Production
if (builder.Environment.IsProduction())
{
    host.UseStartup<Startup>();
}
else
{
    host.UseStartup<Startup>();
}

await host.RunAsync();
```

Phase 3: Deploy .NET API

3.1 Publish the API

```
# Navigate to API folder
cd API

# Publish for Linux
dotnet publish -c Release -r linux-x64 --self-contained true -o ./publish
```

3.2 Create Web App and Deploy

```
# Create web app
az webapp create \
  --name ecom-ai-api \
  --resource-group ecom-ai-rg \
  --plan ecom-ai-plan \
  --runtime "DOTNET|8.0"

# Configure deployment
az webapp config appsettings set \
  --name ecom-ai-api \
  --resource-group ecom-ai-rg \
  --settings ASPNETCORE_ENVIRONMENT=Production

# Deploy using ZIP
az webapp deployment source config-local-git \
  --name ecom-ai-api \
  --resource-group ecom-ai-rg

# Get deployment URL
DEPLOYMENT_URL=$(az webapp deployment source show --name ecom-ai-api --resource-group ecom-ai-rg --query "url" -o tsv)

# Push your code
git remote add azure $DEPLOYMENT_URL
git push azure main:master
```

3.3 Alternative: Deploy via Visual Studio

1. Right-click on API/API.csproj
2. Select **Publish**
3. Choose **Azure** → **Azure App Service**
4. Select your subscription and create new App Service
5. Click **Publish**

Phase 4: Deploy Angular Client

4.1 Update Angular Configuration

In client/angular.json, update the production baseHref:

```
"configurations": {
  "production": {
    "baseHref": "/",
    "optimization": true,
    "outputHashing": "all",
    "sourceMap": false,
    "namedChunks": false,
    "extractLicenses": true
  }
}
```

4.2 Update API URL

In client/src/environments/environment.prod.ts:

```
export const environment = {
  production: true,
  apiUrl: 'https://ecom-ai-api.azurewebsites.net/'
};
```

4.3 Build and Deploy

```
cd client

# Build for production
ng build --configuration production

# Create Static Web App
az staticwebapp create \
  --name ecom-ai-client \
  --resource-group ecom-ai-rg \
  --location eastus \
  --source "./dist/your-app-name" \
  --target "./dist/your-app-name" \
  --api-location "./api"

# Get deployment token
az staticwebapp show \
  --name ecom-ai-client \
  --resource-group ecom-ai-rg \
  --query "properties.provisioningState"
```

Phase 5: Database Migration

5.1 Run EF Core Migrations

```
# Update connection string in appsettings.Production.json first
# Then run migrations

cd API

dotnet ef database update \
  --project Infrastructure/Infrastructure.csproj \
  --startup-project API/API.csproj \
  --context StoreContext \
  --connection "Server=ecom-ai-db.postgres.database.azure.com;Port=5432;User Id=appuser; Password=YOUR_PASSWORD; Database=skinet;SSL
Mode=Require"
```

5.2 Seed Initial Data

```
# Sync products to Azure Search
dotnet run --task sync-products

# Sync RAG documents to Azure Search
dotnet run --task sync-rag
```

Phase 6: Verify Deployment

6.1 Health Checks

Endpoint	Expected Response
https://ecom-ai-api.azurewebsites.net/	HTML response
https://ecom-ai-api.azurewebsites.net/swagger	Swagger UI
https://ecom-ai-client.azurewebsites.net/	Angular app

6.2 Test AI Chat

```
# Test the AI endpoint
curl -X POST "https://ecom-ai-api.azurewebsites.net/api/ai/chat" \
  -H "Content-Type: application/json" \
  -d '{"message": "What is your return policy?"}'
```

Cost Estimation (Monthly)

Service	Tier	Estimated Cost
App Service (Linux)	B1	~\$13/month
PostgreSQL Flexible	Standard_B1ms	~\$20/month
Azure Cache for Redis	Basic (C0)	~\$30/month
Azure AI Search	Standard	~\$50/month
Static Web Apps	Free	\$0
Bandwidth	~10GB	~\$5

Total Estimated: ~\$118/month

Troubleshooting

Common Issues

Issue	Solution
500 Internal Server Error	Check App Service logs in Azure Portal
Database connection failed	Verify SSL Mode=Require in connection string
Redis connection failed	Ensure SSL=True and correct port 6380
CORS errors	Update AllowedOrigins in appsettings
AI search not working	Verify Azure Search API key and endpoint

View Logs

```
# Stream logs from App Service
az webapp log tail \
  --name ecom-ai-api \
  --resource-group ecom-ai-rg
```

Security Checklist

- ☐ Change default JWT signing key
- ☐ Enable HTTPS only
- ☐ Configure CORS properly
- ☐ Store secrets in Azure Key Vault
- ☐ Enable Azure AD authentication (optional)
- ☐ Set up firewall rules for database
- ☐ Enable diagnostic logging

Next Steps

1. **Set up CI/CD** with GitHub Actions
2. **Configure custom domain** with SSL
3. **Enable Azure Monitor** for alerting
4. **Set up Azure Key Vault** for secrets
5. **Configure backup** for PostgreSQL

Generated by GitHub Copilot for eCOM-AI